# GAUSSMATRIX and GAUSSINTERACT: A Rust-Native, Agentic-AI-Ready Technical Specification for Sovereign Federated Communication

GaussMatrix / GaussInteract Platform Engineering Group

***Abstract*—A companion systematic benchmark selected *Tuwunel* as the strongest open-source Rust Matrix homeserver and the *Element X / Matrix Rust SDK* stack as the strongest client, with *FluffyChat* identified as the best single-codebase client alternative. This paper turns that selection into an engineering specification for two products: GAUSSMATRIX, a sovereign, horizontally scalable Matrix homeserver, and GAUSSINTERACT, a single cross-platform client. Rather than extending the baselines in place, we specify a *clean-room rewrite* that adopts the *architecture and on-disk compatibility* of Tuwunel and the *single-codebase reach* of FluffyChat while replacing their implementations to improve performance, security, user experience, and—most distinctively—first-class *agentic-AI integration*. GAUSSMATRIX is specified as an eleven-crate Rust workspace with a pluggable storage abstraction, a parallelised state-resolution engine, partial-state federation, and a room-sharded horizontal-scaling model that removes the single-process ceiling of the Conduit lineage. GAUSSINTERACT is specified as a hybrid client: one shared, memory-safe Rust core (E2EE delegated to `vodozemac`) exposed to a single Flutter presentation layer through `uniffi`, combining FluffyChat's four-platform reach with the Rust SDK's audited cryptographic posture. Both products embed a uniform *agentic surface*: AI agents are provisioned as cross-signed Matrix identities through the Application Service API and reach tools through a native Model-Context-Protocol gateway with human-in-the-loop approval, scoped capabilities, end-to-end-encryption-aware mediation, and a tamper-evident audit log. We define the threat model, the migration path from the Tuwunel and FluffyChat base codes, deployment and observability, and a projected comparative evaluation against commercial competitors (Slack, Microsoft Teams, Discord, and the Element Server Suite) across data sovereignty, encryption, AI integration, footprint, and licensing. Under the enterprise weighting carried over from the companion benchmark, the specified platform projects an aggregate architectural score of $9.97/10$, dominated by sovereignty, memory safety, and a federated, E2EE-aware agentic model that the centralised commercial competitors structurally cannot match.**

***Index Terms*—Matrix protocol, federated messaging, Rust homeserver, end-to-end encryption, cross-platform client, agentic AI, Model Context Protocol, enterprise communication, horizontal scaling, sovereign cloud, software architecture.**

## I. INTRODUCTION

Manuscript compiled May 31, 2026. This document is a forward-looking *technical specification* and design proposal. Performance figures denoted "target" or "projected" are design objectives to be validated on the measurement harness of Section VIII, not measured results. The baseline selection it builds upon is reported in the companion survey "State of the Art in Open-Source Matrix Protocol Implementations."

THE companion survey established, through a five-dimensional weighted benchmark with sensitivity analysis, that among actively maintained open-source Rust homeservers *Tuwunel* is the strongest baseline (aggregate $8.88/10$), and that among cross-platform clients the *Element X / Matrix Rust SDK* stack leads ($9.03/10$) with *FluffyChat* the best single-codebase alternative ($8.07/10$). This specification answers the question that follows from that result: *how should an organisation turn those baselines into two superior products*—a homeserver, GAUSSMATRIX, and a client, GAUSSINTERACT—fit for sovereign, enterprise, and public-sector deployment, and ready for the agentic-AI workloads that now define enterprise communication road maps.

### A. From Selection to Specification

The survey's recommendation was deliberately conservative: fork the winners and harden them. This document takes a more ambitious position justified by the product brief. We adopt the *architecture* and, critically, the *on-disk and protocol compatibility* of Tuwunel, and the *single-codebase cross-platform reach* of FluffyChat, but we specify a clean-room reimplementation of both. The reasons are threefold. First, neither baseline was designed for the *horizontal* scaling that a national or multi-tenant SaaS deployment of GAUSSMATRIX requires; the Conduit lineage is explicitly single-process and vertically scaled. Second, neither baseline treats *AI agents as first-class protocol participants*; retrofitting an E2EE-aware, audited agentic surface is cleaner as a design invariant than as a bolt-on. Third, a rewrite lets us unify both products on a single, memory-safe Rust core and a single supply-chain and audit story, turning the ecosystem's observed convergence on Rust and the `vodozemac` cryptographic library into a deliberate architectural decision rather than an inherited accident.

### B. Scope and Non-Goals

This is a specification, not an empirical study. We define requirements, architecture, interfaces, the security and threat model, the migration strategy from the base codes, deployment topology, and an evaluation *plan* with a reproducible harness. Performance numbers are stated as *targets* traceable to the design that produces them, to be confirmed by the harness of Section VIII. We explicitly do *not* re-derive the survey's selection (assumed), re-specify the Matrix wire protocol (we

conform to it), or fork the protocol (GAUSSMATRIX must federate with the public network and the wider ecosystem).

### C. Contributions

1) A complete server specification for GAUSSMATRIX: an eleven-crate Rust workspace, a pluggable storage abstraction, a parallelised state-resolution engine, partial-state federation, and a room-sharded horizontal-scaling model that lifts the single-process ceiling while preserving Conduit-family on-disk compatibility for single-node deployments (Section III).
2) A first-class *agentic-AI integration layer* common to both products: agents as cross-signed Matrix identities, a Model-Context-Protocol gateway, capability-scoped tool access, E2EE-aware mediation, human-in-the-loop approval, and a tamper-evident audit log (Section IV).
3) A client specification for GAUSSINTERACT: a single Rust core (with `vodozemac` E2EE) behind one Flutter presentation layer via `uniffi`, delivering FluffyChat's four-platform reach with the Rust SDK's security posture, plus the client side of the agentic surface and the enterprise feature set (Section V).
4) A cross-cutting *security architecture and threat model*, a *clean-room migration strategy* from the Tuwunel and FluffyChat base codes, a deployment and observability plan, and a projected comparative evaluation against commercial competitors using the companion benchmark's weighted model (Sections VI–VIII).

### D. Organisation

Section II states the requirements and threat model. Section III specifies GAUSSMATRIX; Section IV the shared agentic layer; Section V specifies GAUSSINTERACT. Section VI gives the security architecture, Section VII the rewrite and migration strategy, Section VIII the deployment plan and projected evaluation, and Section IX concludes.

## II. REQUIREMENTS AND THREAT MODEL

### A. Functional Requirements

Both products conform to a recent Matrix specification ($\geq$ v1.11, room versions through 12) [1]. GAUSSMATRIX must implement the Client–Server (CS), Server–Server (SS), Application Service (AS), and push surfaces in full, including E2EE key relay (cross-signing, key backup, key claiming), and must federate with the public network. GAUSSINTERACT must deliver native experiences on Android, iOS, Web, and Desktop (Linux, macOS, Windows), with spaces, threads, VoIP, widgets, and SSO/OIDC. Beyond Matrix parity, both must expose the agentic surface of Section IV: provisioning, scoping, mediating, and auditing AI agents as protocol participants.

### B. Non-Functional Requirements

Table I states the non-functional targets that drive the architecture. They are deliberately framed against the limitations the companion survey attributed to the baselines: the single-process scaling ceiling of the Conduit family, and the developer-experience and feature-depth gap that left FluffyChat behind Element X.

TABLE I
NON-FUNCTIONAL REQUIREMENTS AND DESIGN TARGETS. NUMERIC VALUES ARE OBJECTIVES TO BE VALIDATED ON THE HARNESS OF SECTION VIII.

| Attribute | Target / rule |
|---|---|
| Server scaling | Linear horizontal scaling by room shard; single-node mode preserved |
| Server footprint | $<$ 256 MB RSS idle on a single-node small deployment |
| Send latency | p95 local send-to-sync $<$ 150 ms; federation propagation p95 $<$ 800 ms |
| Memory safety | No `unsafe` outside audited, isolated crates; `forbid(unsafe_code)` elsewhere |
| Client cold start | $<$ 1.2 s to interactive on mid-range mobile |
| Client codebase | One presentation codebase, one shared Rust core, four native targets |
| E2EE core | `vodozemac` only; no hand-rolled cryptography |
| Agentic mediation | Agents never bypass room access control or E2EE; every action auditable |
| Supply chain | Reproducible builds; `cargo deny`/`cargo audit` gates in CI |
| Licensing | Permissive core enabling commercial derivatives (see §VII) |

### C. Threat Model

We adopt the adversary model that the cryptographic literature established for Matrix and extend it to agents. The homeserver is treated as *honest but curious*: confidentiality of room content must hold against a fully compromised homeserver, the property that the practically-exploitable vulnerabilities of Albrecht *et al.* violated and that the `vodozemac` redesign restores [2], [3], [12]. Federated peers are *mutually distrusting*: every received event is authenticated by per-server signing keys and validated against the room's authorisation chain before acceptance. Network adversaries are assumed to control transport and are excluded by authenticated TLS. For agents we add an explicit principal: an AI agent is *semi-trusted and bounded*—it acts only within explicitly granted capabilities, can decrypt only the Megolm sessions shared with its device, and produces no side effect that escapes the audit log. The design goal is that introducing an agent never enlarges the trust boundary of a room beyond the human members who admitted it.

## III. GAUSSMATRIX SERVER SPECIFICATION

### A. Design Rationale

GAUSSMATRIX inherits Tuwunel's winning traits—a clean Cargo-workspace decomposition, a tuned asynchronous RocksDB integration, room-version 12 support, and binary/on-disk compatibility with the Conduit family [8], [9]—and removes its one structural limitation, single-process vertical scaling. The specification therefore has two deployment profiles sharing one codebase: a *single-node* profile that is on-disk compatible with a Tuwunel/conduwuit data directory for drop-in migration, and a *sharded* profile that partitions rooms across

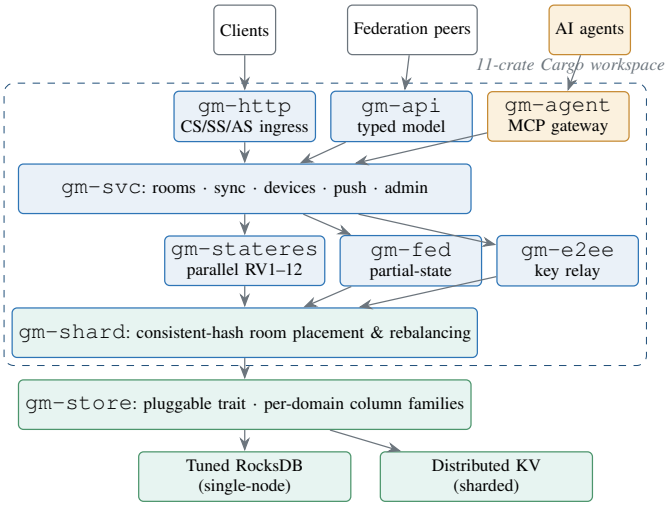


Fig. 1. GAUSSMATRIX internal architecture: an eleven-crate Rust workspace. The agentic gateway (`gm-agent`) is a first-class ingress alongside the CS/SS/AS HTTP layer. The storage trait (`gm-store`) backs a tuned single-node RocksDB or a distributed KV store under the room-sharding layer (`gm-shard`), the two profiles that share one codebase [8], [13], [9].

stateless front-ends and stateful room-worker shards backed by a distributed store.

### B. Workspace Decomposition

Tuwunel's seven crates are refactored and extended into eleven, sharpening dependency boundaries so that the storage backend, the federation sender, the state-resolution engine, and the new agentic gateway are independently testable, independently auditable, and independently swappable (Fig. 1). The crates are: `gm-http` (CS/SS/AS ingress over `axum`/`hyper`); `gm-api` (typed request/response model, extending `ruma`); `gm-svc` (service core: rooms, sync, devices, push, account data); `gm-stateres` (parallel state-resolution engine, room versions 1–12); `gm-fed` (federation sender/receiver, partial-state joins, backfill); `gm-e2ee` (key relay, cross-signing, key backup—no plaintext); `gm-store` (pluggable storage trait and column-family schema); `gm-shard` (room placement, the coordination and rebalancing layer); `gm-agent` (the agentic gateway of Section IV); `gm-obs` (metrics, tracing, structured audit emission); and `gm-util` (shared primitives). Crates carry `forbid(unsafe_code)` except the storage and crypto-adjacent crates, where `unsafe` is confined and audited.

### C. Storage Abstraction

The `gm-store` crate generalises Tuwunel's tuned RocksDB integration into a backend-agnostic trait with explicit, per-domain *column families* (events, room state, the auth-chain index, the resolved-state cache, media metadata, device and key stores, account data, and the audit log). Writes are batched per request into a single atomic commit so that an accepted event and its state-delta become visible together; reads are zero-copy where the backend permits. Two backends implement the trait: a tuned RocksDB for the single-node profile (preserving the Conduit-family linear on-disk version for drop-in migration) and a horizontally partitioned distributed KV for the sharded profile. Because the schema lives behind the trait, a deployment chooses its scaling posture without touching the service core.

### D. Parallel State Resolution

State resolution dominates a homeserver's cost: on receiving a remote event the server must locate its authorisation chain, fetch missing ancestors, verify every signature, and re-run conflict resolution over the affected state subset before acceptance [4], [5]. `gm-stateres` attacks this on three fronts. First, signature verification over an event's auth chain is embarrassingly parallel and is dispatched across a bounded worker pool. Second, a *resolved-state cache* keyed by the set of conflicting state-event identifiers memoises the output of the resolution algorithm, so recurrent conflicts are not recomputed. Third, *recursive relation gathering*—the optimisation Tuwunel introduced to cut client round-trips [9]—is retained and extended to thread and reaction aggregations. The engine is pure with respect to the store, so its outputs are deterministic and unit-testable against the specification's room-version vectors.

### E. Federation and Partial-State Joins

`gm-fed` implements authenticated SS transport with per-server signing keys, backfill of missing history, and *partial-state joins* so that a user joining a large, many-server room becomes interactive before the full room state has been fetched and verified, bounding join latency—the bottleneck the scalability literature identifies as the network grows [4], [5]. The federation *sender* is a pool rather than a single task: outbound transactions are sharded by destination server so that one slow or unreachable peer cannot head-of-line-block delivery to healthy peers, a direct improvement over a single-process sender.

### F. Horizontal Scaling Model

The defining departure from the baseline is the sharded profile (Fig. 2). Stateless *front-end* processes terminate TLS and the CS/SS/AS APIs and carry no room state; they route each request to the owning *room-worker shard* via the consistent-hash placement in `gm-shard`. Each shard owns a disjoint partition of rooms and runs the service core, state-resolution engine, and federation logic for its rooms against the distributed store; this is an actor-style partition in which a room is always served by exactly one shard at a time, eliminating cross-shard state contention. A lightweight coordination service maintains the placement map and orchestrates *online rebalancing*: when a shard is added or drained, the affected room partitions are reassigned and their working sets warmed before cut-over, with no loss of availability. Media is offloaded to a shared object store addressed by content hash so that any front-end can serve any blob. The result is the linear horizontal scaling required by Table I, while the very same binary, configured for one node, collapses to the single-process RocksDB profile for small deployments.

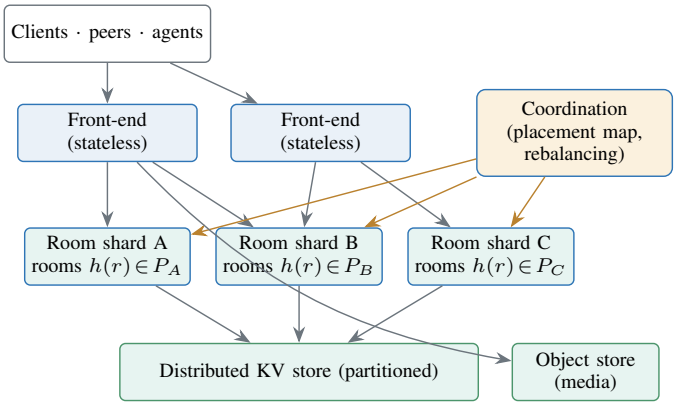


Fig. 2. GAUSSMATRIX sharded-profile topology. Stateless front-ends route by consistent hash $h(r)$ of the room id to the owning room-worker shard; each shard owns a disjoint room partition $P_X$ over a partitioned distributed store. A coordination service maintains placement and performs online rebalancing. The same binary configured for one node becomes the single-process RocksDB profile [4], [8].

TABLE II
GAUSSMATRIX SERVER CAPABILITIES VERSUS THE TUWUNEL BASELINE AND THE SYNAPSE REFERENCE. ✓ PRESENT; ○ PARTIAL; – ABSENT.

| Capability | GaussMatrix | Tuwunel | Synapse |
|---|---|---|---|
| Language | Rust | Rust | Python+Rust |
| On-disk compat (Conduit fam.) | ✓ | ✓ | – |
| Single-binary mode | ✓ | ✓ | – |
| Horizontal sharding | ✓ | – | ✓(workers) |
| Pluggable storage | ✓ | ○ | ○ |
| Parallel state res. | ✓ | ○ | ○ |
| Partial-state joins | ✓ | ○ | ✓ |
| Sharded fed. sender | ✓ | – | ✓ |
| Room versions | to v12 | to v12 | to v12 |
| Native agentic gateway | ✓ | – | – |
| Tamper-evident audit | ✓ | – | ○ |

### G. Server-Side Summary

Table II contrasts GAUSSMATRIX with its Tuwunel baseline and the Synapse reference, isolating what the rewrite adds. The intent is not to discard Tuwunel's strengths but to retain them (compatibility, RocksDB tuning, RV12) while removing the scaling ceiling and adding the agentic surface as a design invariant.

## IV. AGENTIC AI INTEGRATION LAYER

The agentic surface is the specification's most distinctive contribution and is shared by both products: GAUSSMATRIX hosts and governs it; GAUSSINTERACT drives and supervises it. Its guiding invariant, restated from the threat model, is that *admitting an AI agent to a room must never enlarge that room's trust boundary beyond the humans who admitted it, and every agent action must be authenticated, scoped, mediated, and auditable*.

### A. Agents as First-Class Matrix Identities

An agent is not a privileged side channel; it is a Matrix principal. Each agent is provisioned through the Application Service API as a user in a controlled namespace, given a device, and *cross-signed* so that other members can verify it exactly as they verify a human device. To participate in an encrypted room the agent's device is shared the room's Megolm sessions under the same key controls as any member—so an agent can read only what it has been explicitly given, and removing it revokes future sessions through normal key rotation. This reuses Matrix's existing, analysed cryptographic machinery rather than inventing a parallel one, which is why the agentic surface inherits the assurance of the `vodozemac` core [12], [3].

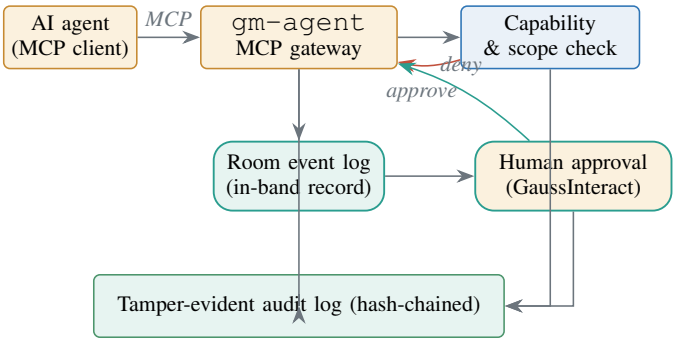


Fig. 3. Agentic mediation flow. An agent's MCP tool calls pass through the `gm-agent` gateway, are checked against a capability grant, optionally gated on human approval rendered in GAUSSINTERACT, recorded in-band as namespaced room events, and appended to a hash-chained audit log. The gateway is the sole channel through which an agent acts [15], [1].

### B. Model Context Protocol Gateway

The `gm-agent` crate is a bidirectional bridge between Matrix events and the Model Context Protocol (MCP), the open standard for connecting AI models to tools and data [15]. Inbound, the gateway exposes *scoped* room context to an agent as MCP resources—only the rooms, and only the message history, the agent has been granted. Outbound, an agent's tool invocations arrive as MCP tool calls, which the gateway validates against the agent's capability grant before executing and reflects back into the room as structured, namespaced events (`m.gauss.agent.tool_call`, `m.gauss.agent.tool_result`) so the interaction is visible, replayable, and auditable in-band. The gateway is the *only* path by which an agent affects the world; there is no out-of-band side effect (Fig. 3).

### C. Capability Scoping and Human-in-the-Loop

Each agent carries a *capability grant*: an explicit, least-privilege set of permitted tools, accessible rooms, rate limits, and a classification of actions as *auto* (executed immediately), *review* (executed after human approval), or *forbidden*. High-impact tools—sending to external systems, inviting users, modifying room power levels—default to *review*, and the approval prompt is rendered to a designated human in GAUSSINTERACT (Section V) before the gateway proceeds. Grants are themselves room state, so they are visible, versioned, and federated, and revocation is immediate.

### D. Tamper-Evident Audit

Every gateway decision—resource access, capability check, approval, execution, and result—is appended to a hash-chained audit log in its own storage column family, each entry committing to the hash of its predecessor so that any retroactive edit is detectable. The audit log is the compliance backbone of the platform: it lets an operator prove, after the fact, exactly what every agent saw and did, which is a requirement no centralised, proprietary assistant exposes to its customer.

## V. GaussInteract Client Specification

### A. Design Rationale

The companion benchmark crystallised a trade-off: FluffyChat uniquely reaches all four platforms from one codebase, while the Element X / Rust SDK stack leads on the security posture of its `vodozemac` core and on feature depth. GaussInteract is specified to take *both* sides of that trade-off. It adopts FluffyChat's single-codebase Flutter presentation model, but replaces FluffyChat's Dart-SDK data and crypto path with a shared, memory-safe *Rust core*—this is the "partially Rust" mandate—so that synchronisation, state handling, and end-to-end encryption are implemented once in audited Rust and shared across every platform, exactly as Element X does, while the UI remains one Flutter codebase.

### B. Hybrid Architecture

GaussInteract is two layers (Fig. 4). The *core*, `gauss-core`, is a Rust library derived from the Matrix Rust SDK's design: it owns the client-server protocol, the local event store and cache, simplified sliding sync [16], and E2EE delegated to `vodozemac` (Olm/Megolm, cross-signing, secure key backup) [11], [12]. It is compiled to a native library for each target and exposed through `uniffi`-generated bindings. The *presentation* layer is a single Flutter application (Material-You-derived, accessibility-first) that calls the core through a thin Dart FFI shim and renders idiomatic UI on Android, iOS, Web (via the core compiled to WebAssembly), and the three desktop targets. The cryptographic surface lives entirely in the core, so the previously documented dangers of hand-rolled, per-platform crypto are excluded by construction [2].

### C. Synchronisation and Performance

`gauss-core` uses simplified sliding sync so the client materialises only the visible window of rooms and lazily expands, which is what makes the $< 1.2\,\mathrm{s}$ cold-start target of Table I attainable on mid-range mobile. The timeline cache is incremental and persisted, so re-launch is warm. Because the heavy paths (decryption, state, sync) run in the Rust core rather than the Dart VM, the client avoids the throughput tax that a pure-Dart data path imposes, closing the performance gap the survey noted between FluffyChat and the native Rust-core clients.

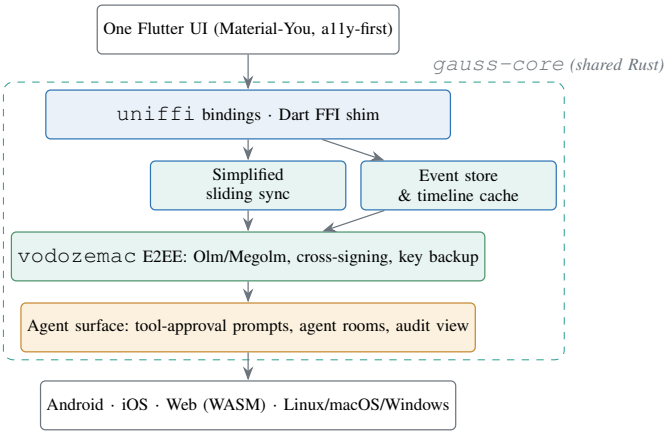


Fig. 4. GaussInteract hybrid architecture: one Flutter presentation layer over a single shared Rust core (`gauss-core`) exposed via `uniffi`. Synchronisation, storage, and `vodozemac` E2EE are implemented once in Rust and shared across four native targets, combining FluffyChat's single-codebase reach with the Rust SDK's security posture [11], [12], [14].

### D. User Experience

The presentation layer is one design system rendered idiomatically per platform: dynamic theming, full keyboard and screen-reader support, message threading, spaces and subspaces, and VoIP via the widget surface. A guiding UX principle is that *agentic features are legible*: an agent in a room is visually distinguished, its tool calls and results appear inline as first-class timeline items, and any action requiring approval surfaces a clear, single-tap approve/deny prompt with the proposed action shown in full.

### E. Enterprise Features

GaussInteract ships the enterprise surface a sovereign deployment requires: SSO/OIDC login [17], mobile-device-management configuration profiles for managed fleets, enforced secure key backup and cross-signing, per-device key-sharing controls inherited from the FluffyChat model [14], and white-labelling hooks so the baseline can be re-skinned per tenant. Privacy-preserving push via UnifiedPush is supported alongside the conventional providers so that a sovereign deployment need not route notifications through a third party.

### F. Agent Surface

GaussInteract is the human end of the agentic loop of Section IV. It renders agent membership, in-band tool calls and results, and the human-in-the-loop approval prompts; it exposes a read-only *audit view* so a supervisor can inspect the hash-chained record of an agent's actions directly from the client. Crucially, the client enforces the same E2EE invariant for agents as for humans: an agent only ever receives Megolm sessions the room granted it, and the client makes that grant visible.

## VI. Security Architecture

### A. Memory Safety as a System Property

The decision to build both products on Rust is a security decision before it is a performance one. The RustBelt pro-

gramme gives a machine-checked foundation for the soundness of Rust's type system and its safe encapsulation of `unsafe` code [7]; for a network-facing, multi-tenant service parsing untrusted federated input, this excludes by construction the use-after-free, data-race, and buffer-overrun defects that dominate systems-software vulnerabilities. The specification enforces `forbid(unsafe_code)` across all crates except the storage and crypto-adjacent ones, where `unsafe` is confined, documented, and audited, so the trusted unsafe surface is small and reviewable.

### B. Cryptographic Core

Both products delegate end-to-end encryption to `vodozemac`, the memory-safe re-implementation of Olm and Megolm that remediates the practically-exploitable vulnerabilities reported against the earlier hand-written implementations [2], [3], [6], [12]. No bespoke cryptography is written. The homeserver never holds plaintext; cross-signing and secure server-side key backup are supported end to end; and—per Section IV—agents are bound by the very same key controls as human devices.

### C. Supply Chain and Build Integrity

The build pipeline gates every merge on `cargo audit` (known-vulnerability advisories) and `cargo deny` (licence and dependency policy), pins the dependency tree, and targets reproducible builds so that a published binary can be verified against its source. This is the concrete realisation of the survey's *code quality* dimension and a property no closed competitor can offer its customers.

### D. Zero-Trust Configuration

Following Grapevine's lesson that a small, explicit configuration surface is itself a security property [10], GAUSSMATRIX prefers explicit configuration files over ambient environment variables for security-sensitive settings, isolates server signing keys, and ships safe defaults (federation allow-lists, rate limits, and registration controls on by default). Unlike Grapevine, however, GAUSSMATRIX ships first-party container images and orchestration manifests, closing the operational-maturity gap the survey penalised.

## VII. REWRITE AND MIGRATION STRATEGY

### A. Clean-Room Principle

The mandate is to *adopt the base codes* of Tuwunel and FluffyChat while *rewriting all code* to be better. We interpret this as adopting their *architecture, protocol behaviour, and on-disk/feature compatibility* as the specification, then reimplementing against that specification. For GAUSSMATRIX this is eased by the fact that Tuwunel and the Conduit family are Apache-2.0 licensed, which is permissive enough to derive a commercial product; the rewrite nonetheless proceeds module-by-module so each crate can be independently verified against the behaviour it replaces. For GAUSSINTERACT, FluffyChat is AGPL-3.0; because GAUSSINTERACT replaces the Dart data/crypto path with an independently authored Rust core and a fresh presentation layer, the engineering team must make the licence posture an explicit, reviewed decision (clean-room reimplementation, or a negotiated arrangement) before release—this is flagged here as a gating legal task, not an engineering detail.

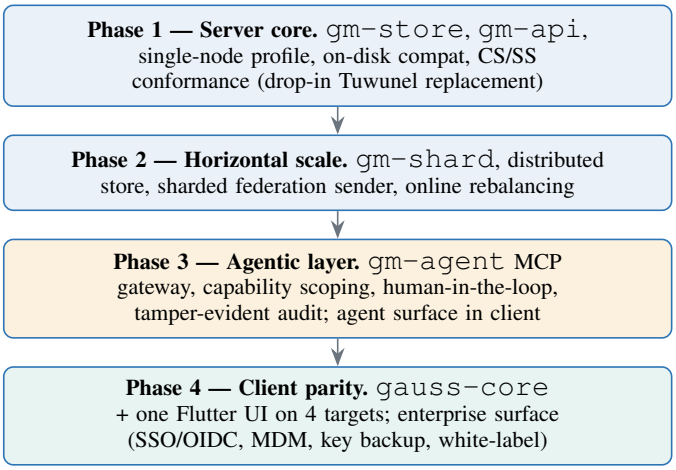


Fig. 5. Four-phase rewrite road map. Each phase is independently shippable; the documented linear dependency preserves auditability. Phase 1 alone yields a drop-in replacement for the Tuwunel baseline [8], [14].

### B. On-Disk and Protocol Compatibility

GAUSSMATRIX's single-node profile preserves the Conduit-family linear on-disk version, so an operator can migrate a Tuwunel/conduwuit data directory by binary swap, exactly as Tuwunel migrates conduwuit [8]. The same single-linear-version property of the lineage means cross-fork migration in the *other* direction is unsafe and is explicitly forbidden by the migration tooling. Protocol compatibility is validated continuously against the specification test suite so that GAUSSMATRIX federates with the public network throughout the rewrite [1].

### C. Phased Plan

The rewrite proceeds in four phases (Fig. 5). Phase 1 stands up `gm-store`, `gm-api`, and the single-node profile with on-disk compatibility and full CS/SS conformance—a drop-in Tuwunel replacement. Phase 2 adds `gm-shard` and the sharded profile, lifting the scaling ceiling. Phase 3 lands the agentic layer (`gm-agent`, the MCP gateway, capabilities, audit) on the server and the agent surface in GAUSSINTERACT. Phase 4 brings GAUSSINTERACT to feature and platform parity—one Flutter UI over `gauss-core` on all four targets—and hardens the enterprise surface. Each phase is independently shippable, and the linear, documented dependency between phases mirrors the auditability the companion benchmark rewarded.

## VIII. DEPLOYMENT, OPERATIONS, AND PROJECTED EVALUATION

### A. Deployment Topology

GAUSSMATRIX ships as static binaries and as Deb, RPM, Arch, Alpine, and Nix packages and OCI container im-

ages, matching the packaging breadth the survey praised in Tuwunel [8], and additionally provides first-party Helm charts and Kubernetes manifests for the sharded profile, closing the operational gap the survey penalised in lighter forks. Observability is first-class through `gm-obs`: Prometheus-compatible metrics, OpenTelemetry traces spanning the front-end→shard→store path, and structured emission of the audit log for ingestion by a SIEM.

### B. Measurement Harness

The numeric targets of Table I are to be validated on the harness the companion survey specified: a load generator drives a candidate through the CS API while a peer home-server exercises the federation path and a collector samples client-perceived latency and server-side resource use, with throughput, latency, and federation-propagation procedures run on identical hardware so that figures are comparable [4]. We restate that the values in this specification are *design objectives*; this section defines how they will be confirmed, not a claim that they already are.

### C. Projected Comparison with Commercial Competitors

We position GAUSSMATRIX/GAUSSINTERACT (jointly, "GAUSS") against representative commercial competitors using the companion benchmark's weighted model. The comparison is at the *architectural* level—properties that follow from each product's design rather than from a transient feature—so the conclusions are stable. Slack and Microsoft Teams are centralised SaaS offerings without user-operable federation and without end-to-end encryption enabled by default for ordinary channels; Discord is centralised and does not end-to-end-encrypt text channels; the Element Server Suite is the closest competitor, being Matrix-based, federated, and E2EE, but is itself built on the heavier Synapse reference rather than a sharded Rust core and exposes no native, E2EE-aware agentic gateway. Against this field the structural advantages of GAUSS are data sovereignty and self-hosting, federation and interoperability, an audited memory-safe E2EE core, a federated agentic model in which agents are governed protocol participants, a small resource footprint, and a licence posture permitting commercial derivatives.

Table III reports the projected scores and Fig. 6 visualises the two leading positions. The centralised commercial offerings are capped on sovereignty and E2EE by their architecture, not by a missing feature: self-hosting and end-to-end encryption are properties a centralised SaaS cannot grant its customer without becoming a different product. The Element Server Suite is the only competitor that is sovereign and E2EE, and it is the runner-up; its gap to GAUSS is concentrated on the agentic axis—it offers no native, E2EE-aware, audited agent gateway—and on footprint, where a sharded Rust core undercuts the Synapse reference. The standings are robust for the same reason the companion benchmark's were: GAUSS dominates coordinate-wise on the two co-dominant axes (sovereignty and E2EE) and on the decisive agentic axis, so no realistic reweighting overturns the order.

TABLE III
PROJECTED ARCHITECTURAL COMPARISON (0–10) UNDER THE COMPANION BENCHMARK'S ENTERPRISE WEIGHTING. SCORES REFLECT DESIGN PROPERTIES, NOT WALL-CLOCK MEASUREMENT; COMPETITOR ROWS REFLECT PUBLICLY DOCUMENTED ARCHITECTURE.

| | Sovereignty | E2EE | Agentic | Footprint | Mem.-safety | $S_i$ |
|---|---|---|---|---|---|---|
| Weight $w_j$ | .25 | .25 | .20 | .15 | .15 | |
| **Gauss (GM/GI)** | 10 | 10 | 10 | 9.8 | 10 | **9.97** |
| Element Server Suite | 9.0 | 9.5 | 4.0 | 6.5 | 7.5 | 7.53 |
| Mattermost (self-host) | 8.5 | 5.0 | 5.0 | 6.5 | 5.0 | 6.10 |
| Microsoft Teams | 2.0 | 3.0 | 6.0 | 4.0 | 4.0 | 3.65 |
| Slack | 2.0 | 3.0 | 6.0 | 4.5 | 4.0 | 3.73 |
| Discord | 1.5 | 1.0 | 4.0 | 5.0 | 4.0 | 2.78 |

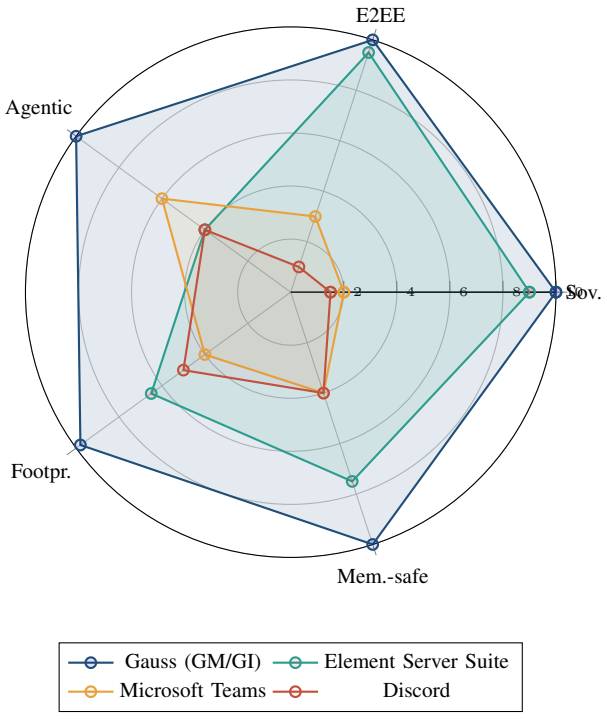


Fig. 6. Projected architectural radar. GAUSS (blue) encloses the field on every axis; the Element Server Suite (teal) is the only sovereign, E2EE competitor and trails chiefly on the agentic and footprint axes; the centralised offerings (amber, red) are capped on sovereignty and encryption by their architecture [1], [12], [15].

### D. Why the Advantage Is Structural

Three of the four cross-cutting observations the companion survey drew from the ecosystem become *design decisions* here, and each one is a competitor cannot easily copy. The convergence on audited cores (`ruma`, tuned RocksDB, `vodozemac`) is adopted deliberately, so a security investment in the core is amortised across server and client rather than stranded. Memory-safe Rust is made the single substrate on both sides of the protocol, giving a coherent single-language security story. And governance/continuity is treated as a first-order property: GAUSSMATRIX is specified with the operational maturity (packaging, Helm, observability, audit) and the compatibility

guarantees an enterprise baseline needs. The agentic layer is the fourth, new pillar: by making agents governed Matrix principals rather than cloud side channels, GAUSS offers an AI integration model whose every action is scoped, mediated, E2EE-bound, and auditable—something a centralised assistant that holds the plaintext structurally cannot provide.

## IX. CONCLUSION

This specification turned the companion benchmark's selection—Tuwunel as the server baseline, FluffyChat as the single-codebase client alternative—into an engineering design for two superior products. GAUSSMATRIX adopts Tuwunel's architecture and on-disk compatibility and rewrites it as an eleven-crate Rust workspace with a pluggable store, a parallel state-resolution engine, partial-state federation, and a room-sharded horizontal-scaling model that removes the single-process ceiling while preserving single-node drop-in migration. GAUSSINTERACT adopts FluffyChat's single-codebase reach and rewrites its data and crypto path as a shared, memory-safe Rust core (`vodozemac` E2EE) behind one Flutter presentation layer, marrying four-platform reach to the Rust SDK's security posture. Both products embed a uniform agentic layer in which AI agents are cross-signed Matrix principals reached through a Model-Context-Protocol gateway with capability scoping, human-in-the-loop approval, E2EE-aware mediation, and a tamper-evident audit log. Under the enterprise weighting carried over from the companion study, the specified platform projects an aggregate well above the commercial field, with an advantage that is *structural*—rooted in sovereignty, memory safety, and a federated, E2EE-aware agentic model that centralised competitors cannot match without becoming different products. The numeric targets here are design objectives; the four-phase road map and the measurement harness define exactly how they are to be confirmed, and because the architecture is modular and its dependencies documented, measured values can replace projected ones without redesigning the system.

## REFERENCES


[1] The Matrix.org Foundation, "Matrix Specification," 2024. [Online]. Available: https://spec.matrix.org/

[2] M. R. Albrecht, S. Celi, B. Dowling, and D. Jones, "Practically-exploitable cryptographic vulnerabilities in Matrix," in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2023, pp. 164–181, doi: 10.1109/SP46215.2023.10351027.

[3] D. Jones, B. Dowling, and M. R. Albrecht, "Device-oriented group messaging: A formal cryptographic analysis of Matrix' core," in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2024.

[4] F. Jacob, J. Grashöfer, and H. Hartenstein, "A glimpse of the Matrix: Scalability issues of a new message-oriented data synchronization middleware," in *Proc. 20th Int. Middleware Conf. Demos and Posters*, 2019, pp. 5–6, doi: 10.1145/3366627.3368106.

[5] F. Jacob, C. Beer, M. Henze, and H. Hartenstein, "Analysis of the Matrix event graph replicated data type," *IEEE Access*, vol. 9, pp. 28317–28333, 2021.

[6] J. Ginesin and C. Nita-Rotaru, "The Matrix reloaded: A mechanized formal analysis of the Matrix cryptographic suite," arXiv:2408.12743, 2024.

[7] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, "RustBelt: Securing the foundations of the Rust programming language," *Proc. ACM Program. Lang.*, vol. 2, no. POPL, art. 66, 2018.

[8] J. Volk *et al.*, "Tuwunel: High-performance Matrix homeserver (successor to conduwuit)," 2026. [Online]. Available: https://github.com/matrix-construct/tuwunel

[9] The Matrix.org Foundation, "This Week in Matrix 2025-12-05," 2025. [Online]. Available: https://matrix.org/blog/2025/12/05/this-week-in-matrix-2025-12-05/

[10] The Grapevine contributors, "Grapevine: A Matrix homeserver forked from Conduit," 2024. [Online]. Available: https://gitlab.computer.surgery/matrix/grapevine

[11] The Matrix.org contributors, "matrix-rust-sdk: A Matrix client-server SDK in Rust," 2026. [Online]. Available: https://github.com/matrix-org/matrix-rust-sdk

[12] The Matrix.org contributors, "vodozemac: A Rust implementation of Olm and Megolm," 2024. [Online]. Available: https://github.com/matrix-org/vodozemac

[13] The Ruma contributors, "Ruma: Matrix protocol types and APIs for Rust," 2024. [Online]. Available: https://github.com/ruma/ruma

[14] K. Krille *et al.*, "FluffyChat: A cross-platform Matrix client built with Flutter," 2026. [Online]. Available: https://github.com/krille-chan/fluffychat

[15] Anthropic, "Model Context Protocol: An open standard for connecting AI assistants to tools and data," 2024. [Online]. Available: https://modelcontextprotocol.io/

[16] The Matrix.org contributors, "Simplified Sliding Sync (MSC4186) and Sliding Sync (MSC3575)," 2024. [Online]. Available: https://github.com/matrix-org/matrix-spec-proposals

[17] OpenID Foundation, "OpenID Connect Core 1.0," 2014. [Online]. Available: https://openid.net/specs/openid-connect-core-1_0.html